

AMR Map Editor

Engineering Guide & README

Documentation update: 2026-09-16

Project version referenced by the existing guide: **v0.11.0**

AMR Map Editor is a CAD/GIS-style web application for creating, editing, validating, and exporting navigation maps for Autonomous Mobile Robots (AMRs), ROS2, Fleet Manager, and Open-RMF / Traffic Editor workflows.

This README consolidates the original engineering guide and the latest implemented export / RMF features.

1. Highlights

- Import ROS occupancy maps from map.yaml + map.pgm / PNG.
- Edit occupancy raster with Freehand, Line, Rectangle, Polygon, and Eraser tools.
- Create navigation objects: Waypoint, Home, Charging Station, Docking Station, Pickup, Drop-off, Waiting, and Parking.
- Create Normal, Preferred, One-way, Bidirectional, and Restricted paths.
- Create No-Go, Slow, Restricted, Parking, Loading, Unloading, Human Traffic, and Safety zones.
- Create RMF building geometry: Wall, Door, Floor Area, Model, and Measurement.
- Configure robot footprint, safety margin, clearance, turning radius, and max speed.
- Validate navigation geometry and topology.
- Export ROS maps, navigation JSON, GeoJSON, RMF .building.yaml, RMF Bundle, and reference-coordinate YAML.

2. Quick Start

npm install

npm run dev

Production build:

npm run build

Tests:

npm test

Typical workflow:

Import Map

-> Clean / Edit Occupancy Map

-> Add Navigation Objects

- > Create Paths
- > Create Zones
- > Create / Verify RMF Floor Geometry
- > Configure Robot
- > Preview
- > Validate
- > Save
- > Export

3. Coordinate Conventions

The editor uses several coordinate spaces.

Space	Purpose
Screen	Browser / pointer interaction
Pixel	Occupancy image pixels
World	ROS / robot map coordinates in meters
RMF reference image	Traffic Editor .building.yaml geometry

Navigation objects, paths, zones, and building geometry are internally stored in **world coordinates in meters**.

Heading convention:

0 rad = 0 deg = East (+X)
 +pi/2 rad = 90 deg = North (+Y)
 pi rad = 180 deg = West (-X)
 -pi/2 rad = -90 deg = South (-Y)

RMF reference-image conversion

The RMF building exporter uses the same worldToReferenceImage() conversion for building vertices and automatically generated reference coordinates.

world / robot coordinate

```

|
+--> GeoJSON / robot coordinate
|
+--> worldToReferenceImage(...)
    |
    +--> RMF building vertex
    +--> reference_coordinates.rmf[]
  
```

This guarantees that `rmf[i]` and `robot[i]` represent the **same physical point** in different coordinate systems.

4. Navigation Objects

After creating an object, the Properties panel supports:

- Name
- ID
- Type
- X / Y position
- Heading in degrees
- Yaw in radians
- Description
- Enabled state

Pickup

Pickup objects include the additional RMF property `pickup_dispenser`.

`pickup_dispenser: [1, "dispenser_name"]`

Drop-off

Drop-off objects include `dropoff_ingestor`.

`dropoff_ingestor: [1, "ingestor_name"]`

If the dedicated field is empty, the exporter may fall back to the object name / ID according to the current exporter logic.

5. Paths: Travel Direction vs Lane Orientation

These are separate concepts.

Travel Direction

Controls whether the lane can be traversed one way or both ways.

One-way: A ----> B

Bidirectional A <----> B

In RMF lane parameters:

`bidirectional: [4, false] # one-way`

or:

`bidirectional: [4, true] # two-way`

Lane Orientation

Controls the robot heading constraint while traversing the lane.

Available values:

- None
- Forward
- Backward

Exported as:

orientation: [1, ""]

orientation: [1, forward]

orientation: [1, backward]

The map canvas displays a dedicated orientation arrow when forward or backward is set.

Travel direction is not the same as robot orientation. A two-way lane can still carry an orientation constraint.

6. Path Connectivity

A path visually crossing another path is **not automatically connected**. True connectivity requires shared topology / coordinates.

The editor supports:

- Endpoint snapping to navigation objects.
- Endpoint snapping to existing path vertices.
- Endpoint snapping to path segments.
- Automatic junction insertion when dropping an endpoint onto the middle of another path.
- Start / End connection status.
- Connect Start / End / Both.
- Merge with a compatible path.
- Reverse Direction / Reverse Vertices.

Green CONNECTED markers indicate actual topology connections rather than only visual overlap.

7. Building Geometry

Wall

Supports Start / End X,Y, Length, Angle, Resize Anchor, Texture, Height / Width, Texture Scale, Alpha, and Enabled.

Door

Supports Door Type, Motion Axis, Motion Degrees, Motion Direction, Plugin, and Right / Left Ratio.

Floor Area

Floor polygons are used by the RMF building exporter and are also the source for **automatic reference-coordinate generation**.

Model

Supports model name, pose, static state, and dispensable state.

Measurement

Used for scale / distance metadata in RMF export.

8. RMF .building.yaml Export

The current exporter targets Traffic Editor style:

`coordinate_system:` `reference_image`

Main structure:

`coordinate_system:` `reference_image`

`crowd_sim:` ...

`graphs:` {}

`levels:`

 L1:

`constraints:` []

`drawing:`

`filename:` `<map>.png`

`elevation:` 0

`features:` []

`floors:` ...

`lanes:` ...

`layers:` ...

`measurements:` ...

`vertices:` ...

`walls:` ...

`lifts:` {}

`name:` `<map_name>`

The exporter converts internal world-meter geometry into reference-image coordinates using map resolution, ROS origin, origin yaw, and image Y-axis conversion.

Keep the exported .building.yaml and referenced PNG together.

9. GeoJSON Export

GeoJSON export follows the current graph-oriented reference format.

```
{
  "crs": {
    "type": "name",
    "properties": {
      "name": "urn:ogc:def:crs:EPSG::3857"
    }
  },
  "type": "FeatureCollection",
  "name": "graph",
  "features": []
}
```

Graph points are exported as Point features. Directed path edges use id, startid, endid, cost, and overridable.

A bidirectional path creates forward and reverse edge records. A one-way path creates only the stored forward edge.

10. Automatic RMF <-> Robot Reference Coordinates

Reference coordinates are generated automatically. Manual coordinate entry is not required.

Source

The exporter:

1. Finds enabled Floor polygons with at least 4 unique vertices.
2. Selects the largest enabled Floor as the reference geometry.
3. Chooses exactly **4 principal outer corners**.
4. Uses the same four physical points for both coordinate systems.

Export order:

1. Top-left
2. Top-right
3. Bottom-right
4. Bottom-left

robot[] contains the Floor/world coordinates used on the robot / GeoJSON side.

rmf[] contains the same points after worldToReferenceImage() conversion.

Therefore:

```
rmf[0] <-> robot[0] Top-left
rmf[1] <-> robot[1] Top-right
rmf[2] <-> robot[2] Bottom-right
rmf[3] <-> robot[3] Bottom-left
```

Output format

The exporter intentionally writes each coordinate pair in compact flow style:

reference_coordinates:

automation_room1:

rmf:

```
- [-0.257426175713, -0.234604896665]
- [159.950112688115, -0.131773041364]
- [159.924785241558, 96.868364927862]
- [-0.063116205376, 96.868211585931]
```

robot:

```
- [-5.192871308786, 3.331730244833]
- [2.817505634406, 3.326588652068]
- [2.816239262078, -1.523418246393]
- [-5.183155810269, -1.523410579297]
```

The numeric values above are examples only. The application generates values from the current map.

The compact [x, y] writer is used intentionally instead of generic block-array YAML formatting.

11. RMF Bundle

The RMF Bundle can include:

```
<name>.building.yaml
<name>.png
<name>-navigation.json
<name>-reference-coordinates.yaml
README.txt
```

If a valid reference Floor is not available, the bundle can still be exported without the reference-coordinate file.

<name>-navigation.json preserves robot pose / navigation data, including object x, y, yaw, and heading information used by Fleet Manager / Nav2 integrations.

12. Export Formats

Output	Use
.amrmap	Reopen and continue editing a full project
ROS Map ZIP	map.yaml + occupancy map

Output	Use
Occupancy PNG	Preview / RMF reference image
PGM	ROS occupancy map
YAML	ROS map metadata
Waypoints JSON	Navigation objects
Paths JSON	Navigation paths
Zones JSON	Zone definitions
Navigation JSON	Objects + paths + zones + robot config
GeoJSON	Graph / coordinate exchange
.building.yaml	Open-RMF / Traffic Editor
Reference Coordinates YAML	RMF <-> robot coordinate correspondence
RMF Bundle ZIP	RMF building + image + navigation + optional reference coordinates

13. Validation

Validation should be run before deployment / export.

Objects

- Outside map
- Inside obstacle
- Duplicate IDs

Paths

- Obstacle intersection
- No-Go intersection
- Narrow path vs robot footprint
- Disconnected path
- Zero-length segments
- Visual crossing without a topology junction

Zones

- Invalid polygon
- Self-intersection
- Outside map
- Conflicting zones

Robot clearance

Checks footprint + safety margin against map restrictions.

14. Save / Open

Projects are saved locally through IndexedDB.

Typical status:

Unsaved

Saving...

Saved locally

Project export:

<name>.amrmap

Use .amrmap to reopen the complete editable project state.

15. Project Structure

src/

```
|— components/
|   |— editor/
|   |   └─ MapCanvas.tsx
|   |— inspector/
|   |   └─ Inspector.tsx
|   |— dialogs/
|   |   └─ ExportDialog.tsx
|   └─ ...
|— geometry/
|   |— pathTopology.ts
|   |— snapEngine.ts
|   └─ ...
|— map-engine/
|   └─ ...
|— models/
|   └─ index.ts
|— services/
|   |— export/
|   |   |— buildingExporter.ts
|   |   |— geojsonExporter.ts
|   |   |— referenceCoordinatesExporter.ts
|   |   |— rmfBundleExporter.ts
|   |   └─ ...
|   |— import/
|   └─ persistence/
|— state/
|   └─ editorStore.ts
```

```
| └─ projectStore.ts
└─ utils/
    └─ files.ts
```

16. Important Exporter Responsibilities

buildingExporter.ts

- Converts world coordinates to RMF reference-image coordinates.
- Generates vertices, lanes, floors, walls, doors, models, and measurements.
- Exports bidirectional.
- Exports lane orientation.
- Exports Pickup / Drop-off RMF parameters.

geojsonExporter.ts

- Exports graph points / path edges in the selected GeoJSON reference format.

referenceCoordinatesExporter.ts

- Selects the reference Floor.
- Selects exactly four principal corners.
- Uses the same four source points for RMF and robot coordinates.
- Outputs compact [x, y] YAML.

rmfBundleExporter.ts

- Packages RMF assets.
- Adds reference-coordinate YAML when valid Floor geometry is available.

17. Troubleshooting

Reference-coordinate YAML still shows block arrays

If you see:

```
-- 10
- 20
```

confirm the active exporter contains:

```
formatPair(...)
return lines.join('\n')
```

Then clear the Vite build cache:

```
rm -rf dist node_modules/.vite
npm run build
npm run dev
```

Correct format:

- [10, 20]

No reference coordinates generated

Verify:

- A Floor exists.
- The Floor is enabled.
- It has at least 4 unique vertices.
- The Floor represents the intended outer reference area.

Lane orientation missing

Verify the selected path has one of:

orientation = ""

orientation = "forward"

orientation = "backward"

and that the building exporter writes:

orientation: [1, forward]

Pickup / Drop-off RMF property missing

Verify the object type and its Properties field:

Pickup -> pickup_dispenser

Dropoff -> dropoff_ingestor

18. Deployment Checklist

- ☐ Correct map resolution.
- ☐ Correct map origin and origin yaw.
- ☐ Navigation objects are not inside obstacles.
- ☐ Path junctions are truly connected.
- ☐ One-way / bidirectional routing is correct.
- ☐ Lane forward / backward orientation is correct.
- ☐ Pickup dispenser names are correct.
- ☐ Drop-off ingestor names are correct.
- ☐ Floor / wall geometry matches the environment.
- ☐ Reference Floor is enabled and has valid outer corners.
- ☐ Reference coordinates contain exactly four physical pairs.
- ☐ .building.yaml and PNG use matching names.
- ☐ Validation has been run.
- ☐ Exported files were tested in the target RMF / robot workflow.

19. Notes

- Travel direction and lane orientation are intentionally separate.
- Reference coordinate values must come from the active map, not from hard-coded examples.
- The four reference pairs correspond to the same four physical map corners in RMF and robot coordinates.
- Traffic Editor, Fleet Manager, and robot-side systems may use different coordinate representations; keep the coordinate conversion centralized in the exporter.